

BITS, PILANI – K. K. BIRLA GOA CAMPUS

Theory of Computation

(CS F51)

Lecture No. 12



Re Cap - Regular Expression to FSA

Main Result

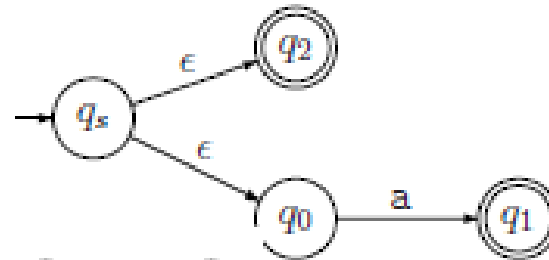
A language is regular if and only if it is accepted by a FSA.

construction to design an NFA accepting the given regular language

Consider the *regular expression* $(a \cup \varepsilon)(aa \cup ba)^*$

The NFA for ε is $\rightarrow (q_2)$ and for a is $\rightarrow (q_0) \xrightarrow{a} (q_1)$.

The NFA for $(a \cup \varepsilon)$ is

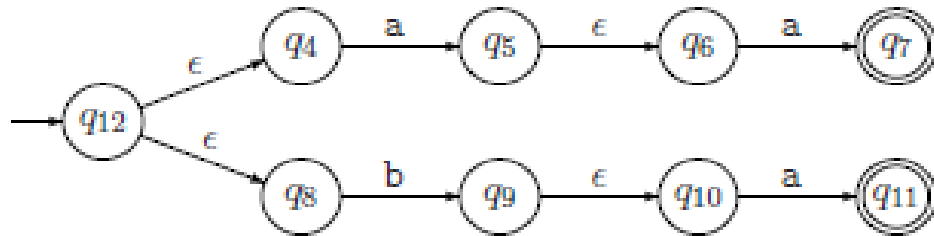


NFA for aa $\rightarrow (q_4) \xrightarrow{a} (q_5) \xrightarrow{\varepsilon} (q_6) \xrightarrow{a} (q_7)$,

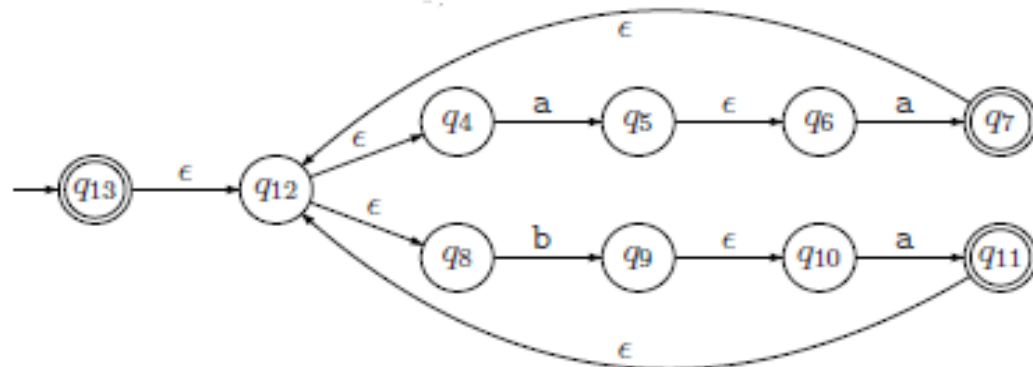
NFA for ba $\rightarrow (q_8) \xrightarrow{b} (q_9) \xrightarrow{\varepsilon} (q_{10}) \xrightarrow{a} (q_{11})$

Re Cap - Regular Expression to FSA

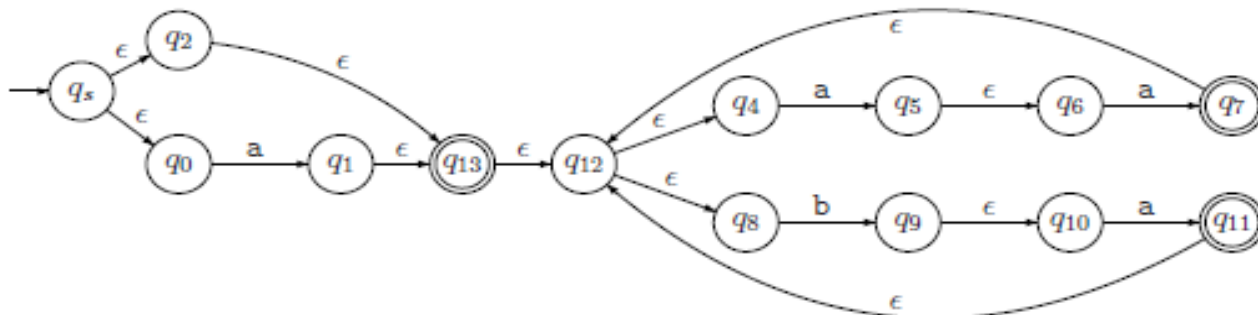
NFA for $aa \cup ba$



NFA for $(aa \cup ba)^*$



NFA for $(a \cup \varepsilon)(aa \cup ba)^*$



Re Cap - FSA to Regular Expression

Conversely, suppose $M = (K, \Sigma, \Delta, s, F)$ is an FSA

We shall construct a *regular expression* R such that $L(R) = L(M)$

$R(i, j, k)$ as the set of all strings in Σ^* that may drive M from state q_i to state q_j *without passing* through any intermediate state numbered $k+1$ or greater.

So $L(M) = \cup \{R(1, j, n) : q_j \in F\}$

Claim: All of $R(i, j, k)$ are *regular* & hence so is $L(M)$.

Proof of the claim is by *induction* on k .

$$R(i, j, k) = R(i, j, k-1) \cup R(i, k, k-1) R(k, k, k-1)^* R(k, j, k-1)$$

Re Cap - FSA to Regular Expression

Observe: When $k = n$, it follows that

$$R(i, j, n) = \{w \in \Sigma^* : (q_i, w) \vdash_M^* (q_j, e)\}$$

$$\therefore L(M) = \cup \{R(1, j, n) : q_j \in F\}$$

Claim: All of $R(i, j, k)$ are *regular* & hence so is $L(M)$.

Proof of the claim is by *induction* on k .

Base Case: $k = 0$

$$R(i, j, 0) = \{a \in \Sigma \cup \{e\} : (q_i, a, q_j) \in \Delta\}$$

Now, each of $R(i, j, 0)$ is finite & so *regular*.

Inductive Hypothesis: Assume $R(i, j, k-1)$ for all i, j are *regular*.

Re Cap - FSA to Regular Expression

Inductive Step: We will show that each set $R(i, j, k)$ can be defined combining previously defined *regular languages* & using the operations of union, concatenation & Kleene Star.

Observe: We can go from q_i to q_j without passing through a state numbered greater than k in either of the following ways:

(1) Go from q_i to q_j without passing through a state numbered greater than $k-1$ **OR**

(2) **a.** q_i to q_k then **b.** from q_k to q_k zero or more times **c.** then q_k to q_j and in each case without passing through a state numbered greater than $k-1$

Therefore

$$R(i, j, k) = R(i, j, k-1) \cup R(i, k, k-1) R(k, k, k-1)^* R(k, j, k-1)$$

FSA to Regular Expression

Example

Consider the NFA

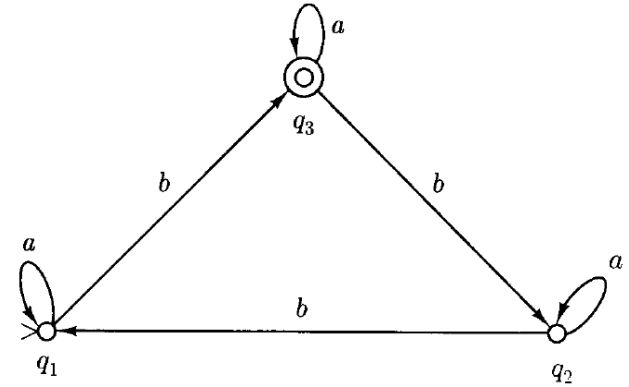
For *simplifying* the construction we make the following *assumptions*:

The FSA M has the following *two properties*:

- (1) M has a single final state = {f}
- (2) If $(q, a, p) \in \Delta$, then $q \neq f$ and $p \neq s$

i.e. there are no transitions into the initial state nor out of the final state

Note: If the given NFA does not have the above property, we can *add* to M a *new initial state s* & a *new final state f* along with *empty transitions* from s to initial state of M & from all final states of M to f such that the new FSA is equivalent to M.



FSA to Regular Expression

Number the states of the FSA as $q_1, q_2, \dots, q_n$ with $s = q_{n-1}$ and $f = q_n$

We want to compute $R(n-1, n, n)$

Construction of $R(n-1, n, n)$

Bottoms Up manner.

i.e, **First** we will compute $R(i, j, 0)$'s

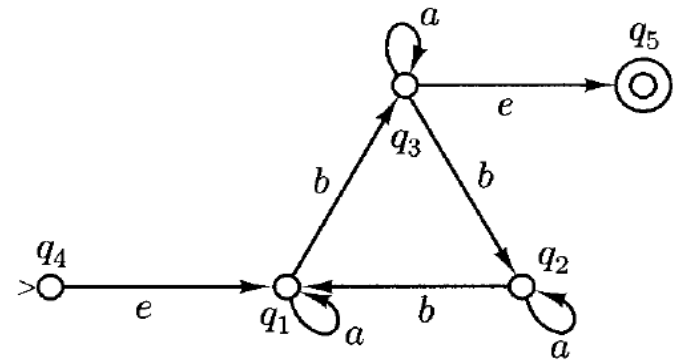
From them the $R(i, j, 1)$'s

And so on

Representation:

At each stage we depict $R(i, j, k)$'s as an **label** for an arrow going from state q_i to state q_j

We **omit** arrows labeled ϕ & self loops labeled $\{e\}$



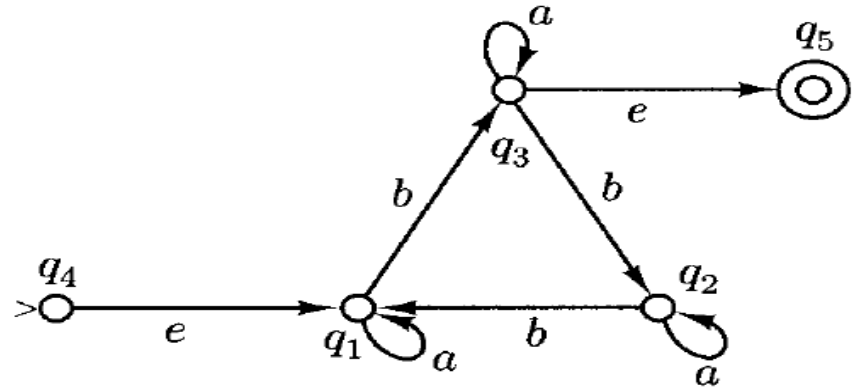
FSA to Regular Expression

Construction:

Compute $R(i, j, 0)$'s

We use the convention

As described earlier to represent it.



Next

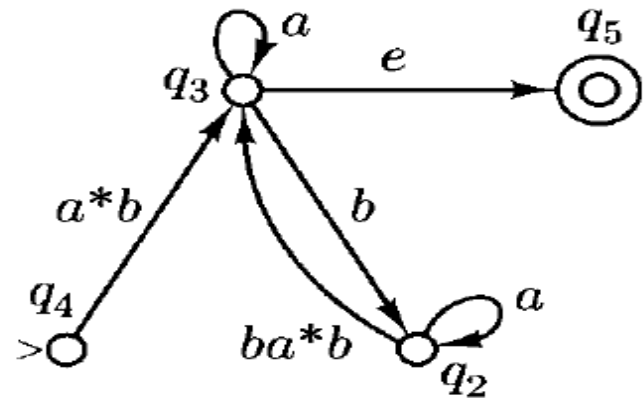
Compute the $R(i, j, 1)$'s

Note: q_1 need not be considered in the rest of construction.

All strings that leads M to

acceptance passing through q_1 have been taken into account in the $R(i, j, 1)$'s.

We say q_1 has been *eliminated*.



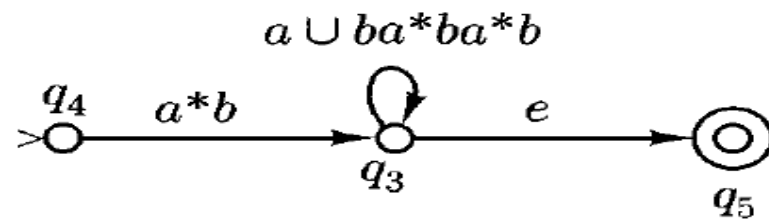
FSA to Regular Expression

Observe:

The resulting FSA has one less state than the original FSA.

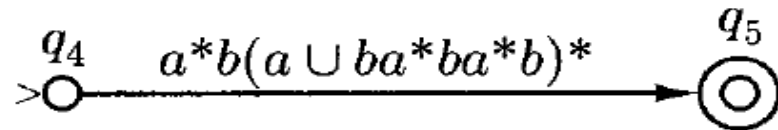
Compute $R(i, j, 2)$'s

q_2 has been *eliminated*.



Compute $R(i, j, 3)$'s

q_3 has been *eliminated*



Note: The FSA has been reduced to a *single transition* from the initial state to the final state

At this point we *terminate* the process & read the label of the transition as the *regular expression* for M.

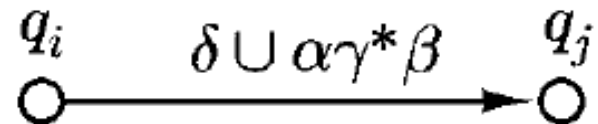
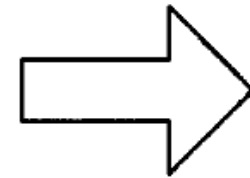
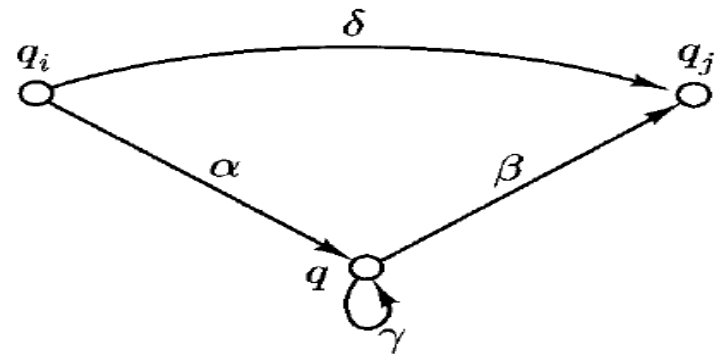
$$R = R(4, 5, 5) = R(4, 5, 3) = a^*b(a \cup ba^*ba^*b)^*$$

FSA to Regular Expression

State elimination process:

Consider the following scenario & we want to *eliminate* q

Eliminating q means *we add label* to the arrow/transition from q_i to q_j with the strings that leads FSA from q_i to q_j while passing via q .



Non Regular Languages

Observe that for a *regular language*,

1. Accepted by a *FSA*

The amount of memory(states) that is needed to determine whether or not a given string is in the language is *finite* and independent of the length of the string

2. Generated by an *regular expression*

If the language consists of an infinite number of strings, then this language should contain infinite subsets having a fairly *repetitive structure*.

Non Regular Languages

Intuitive candidates for *non regular languages*:

1. $L = \{w \in \{0, 1\}^* : w = 0^n 1^n, n \geq 0\}$

This language should be nonregular, because it seems unlikely that a FSA can remember how many 0s it has seen when it has reached the border between the 0's and the 1's.

2. $L = \{1^p : p \text{ is a prime}\}$

Because the prime numbers do not seem to have any repetitive structure that can be used by a regular expression